

Assignment03 PCD part2B

Francesco Carlucci

francesco.carlucci6@studio.unibo.it

Marco Raggini

marco.raggini2@studio.unibo.it

Chapter 1

1.1 Analisi del problema

Il progetto consiste nel realizzare una versione cooperativa e distribuita del Sudoku con un'architettura basata su oggetti distribuiti, in cui i giocatori possono creare nuove griglie, partecipare alla risoluzione di griglie create da altri, entrare ed uscire dinamicamente da qualsiasi griglia. Inoltre i giocatori devono poter vedere in modo consistente le celle della griglia selezionate dagli altri e i valori che vengono inseriti.

1.2 Strategia risolutiva

In questo caso come strategia risolutiva si è deciso di utilizzare un approccio più centralizzato per la gestione dei sudoku attraverso **JavaRMI (Remote Method Invocation)**, utilizzato dai giocatori per trovare le risorse attraverso oggetti remoti.

1.2.1 JavaRMI

JavaRMI è un'API Java che consente a un oggetto in una Java Virtual Machine (JVM) di invocare metodi su un oggetto in un'altra JVM, anche su host diversi.

Il funzionamento di base, che è anche quello utilizzato all'interno di questo progetto, è il seguente:

1. **Registrazione del server:** il launcher crea un oggetto remoto (Server) e lo esporta, registrandolo in un registro RMI.
2. **Recupero dello stub dal client:** il client, in questo caso rappresentato dal **Player**, si connette al registro e recupera il riferimento

all'oggetto remoto, ovvero lo **stub**. Uno stub è un'interfaccia remota e si occupa di effettuare le invocazioni dei metodi remoti.

3. **Invocazione dei metodi:** il player (client) effettua delle chiamate di metodi. Lo stub serializza i parametri e li invia al server, che a sua volta li deserializza, esegue la funzione e ritorna il risultato al client.

1.2.2 Server

Il server è l'entità che gestisce le interazioni tra i player e i sudoku. Al suo interno si possono trovare:

- **Lista dei sudoku:** per tenere traccia di tutti i riferimenti ai sudoku.
- **Lista dei player:** per tenere traccia di tutti i player registrati al server.
- **Mappa sudoku-player:** una mappa che associa ad ogni sudoku i player al suo interno, utilizzata per mandare aggiornamenti in tempo reale solo a chi effettivamente sta giocando ad un determinato sudoku.
- **Metodi di gestione dei sudoku:** sono presenti anche tutti i metodi per la creazione dei sudoku, selezione e deselezione della cella, l'impostazione del valore di una cella e l'entrata e l'uscita dei player dalle partite. Ognuno di questi metodi chiama al suo interno dei metodi per notificare i player dei cambiamenti.

1.2.3 UserCallbackInterface

Il server, per poter notificare i giocatori dei cambiamenti del sudoku effettuati da altri giocatori, chiama i metodi di un'interfaccia (**UserCallbackInterface**) che il player implementa. Essa contiene tre metodi:

- **GetUserId():** ottiene l'id dell'utente;
- **notifyUser(SudokuGrid sudoku):** notifica il player di ogni cambiamento riguardante il sudoku a cui sta giocando;
- **notifySudokuListUpdate(String sudokuId, String creator):** notifica il player quando un nuovo sudoku viene creato in modo che possa aggiungerlo alla sua lista locale;

1.2.4 Architettura MVC

L'applicazione segue un'architettura di tipo **Model–View–Controller (MVC)**, che consente di separare in modo chiaro la logica di gioco, la gestione della comunicazione e l'interfaccia utente. Il **Model** è costituito dalla logica di funzionamento di ciascuna griglia di Sudoku, le informazioni sui giocatori e la gestione della loro comunicazione; il **Controller** gestisce le interazioni dell'utente con la GUI e comunica alla view quando aggiornarsi; infine la **View** si occupa di aggiornare la rappresentazione grafica del gioco in base agli eventi notificati dal controller.